.UBAfiuba
FACULTAD DE INGENIERÍA

Desarrollo de un dispositivo de comunicación de equipos de monitoreo y control energético para la domótica de motorhomes

Esp. Ing. Matías Nahuel Rodriguez

Maestría en Internet de las Cosas

Director: Mg. Lic. Leopoldo Alfredo Zimperz (FIUBA)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Ciudad de [lugar], [mes] de [año]

Resumen

El presente trabajo consiste en el desarrollo de un dispositivo electrónico destinado a integrar equipos del ecosistema Victron Energy a la red de domótica de motorhomes desarrollada por la empresa Enertik Argentina. Su implementación permitirá ampliar las capacidades de la red de domótica existente y posibilitar a los usuarios el monitoreo y control de distintos sistemas del vehículo a través de dispositivos móviles.

Para su desarrollo fue necesario aplicar conocimientos que abarcan la implementación de firmware para microcontroladores, aplicaciones orientadas a Internet de las Cosas destinadas a sistemas Android y el desarrollo de placas de circuitos impresos para productos comerciales.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen I

1. Introducción general 1
1.1. Domótica en motorhomes 1
1.2. Motivaciones del trabajo 1
1.3. Estado del arte 2
1.4. Objetivos y alcances 4

2. Introducción específica 7
2.1. Protocolos de comunicación 7
2.1.1. Protocolo CAN 7
2.1.2. Protocolo Modbus TCP 8
2.1.3. Solicitudes HTTP 8
2.2. Componentes de hardware 9
2.2.1. Microcontrolador ESP32 9
2.2.2. Módulo transceiver 9
2.2.3. Módulo Ethernet W5100 10
2.3. Herramientas de software 11
2.3.1. Entorno de desarrollo Arduino 11
2.3.2. Funcionamiento del ESP32 11
2.3.3. Desarrollo de la aplicación móvil 12

3. Diseño e implementación 15
3.1. Análisis del software 15

4. Ensayos y resultados 17
4.1. Pruebas funcionales del hardware 17

5. Conclusiones 19
5.1. Conclusiones generales 19
5.2. Próximos pasos 19

Bibliografía 21

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control. 3
1.2. Arquitectura de comunicación del sistema con el ESP32 como intermediario entre PRC10, Cerbo GX y la aplicación móvil. 5

2.1. Microcontrolador ESP32. 9
2.2. Funcionamiento del módulo transceiver. 10
2.3. Módulo W5100. 10
2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio. 12

Índice de tablas

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se explica en qué consiste la red de domótica empleada, el estado del arte actual, las motivaciones y objetivos propuestos durante la planificación del trabajo.

1.1. Domótica en motorhomes

Este trabajo fue desarrollado para la empresa Enertik Argentina [1]. Esta empresa se dedica principalmente a la comercialización de equipos asociados al campo de las energías renovables. A su vez, dispone de un área de investigación y desarrollo que se especializa en la fabricación y comercialización de equipos destinados a la domótica de motorhomes.

Cuando se habla de domotizar un espacio, se hace referencia a la incorporación de sistemas electrónicos que permiten automatizar, monitorear y controlar distintos parámetros asociados a una unidad habitacional. Estos sistemas buscan mejorar el confort del usuario, optimizar el uso de la energía y simplificar la operación de los diferentes dispositivos presentes en el entorno.

En el caso particular de los motorhomes, la domótica se aplica al control de diferentes subsistemas del vehículo, tales como la iluminación, los sistemas de audio, el monitoreo del estado de las baterías, el accionamiento de los motores que operan las partes móviles del vehículo y la supervisión de distintos parámetros del sistema energético. La integración de estos elementos permite centralizar la información y el control de los dispositivos, y así facilitar su operación por parte del usuario.

En este contexto, Enertik desarrolla una red de domótica propia destinada a la integración y control de los diferentes dispositivos presentes en un motorhome. Esta red permite interconectar diversos equipos electrónicos y posibilita su supervisión y control mediante una pantalla que permite visualizar y operar los distintos dispositivos conectados a la red de domótica.

1.2. Motivaciones del trabajo

Fueron varias las motivaciones que impulsaron el desarrollo de este trabajo. En primer lugar, dentro de los distintos dispositivos que forman parte de la red de domótica del motorhome, existe un subsistema encargado de monitorear y gestionar la parte energética del vehículo.

Este sistema se compone principalmente de un conjunto de equipos conformado por una batería, un inversor y un cargador.

La batería constituye el dispositivo de almacenamiento de energía del vehículo. A partir de ella se alimentan los diferentes equipos eléctricos presentes en la unidad.

Por su parte, el inversor es el encargado de transformar la tensión continua proveniente de la batería en tensión alterna. Esto permite alimentar dispositivos eléctricos que normalmente funcionan conectados a redes eléctricas domiciliarias.

Finalmente, el cargador tiene la función de convertir la tensión alterna proveniente de la red eléctrica en una forma adecuada para recargar las baterías y almacenar energía en ellas.

En la actualidad se puede observar una tendencia, en donde los fabricantes de motorhomes se encuentran migrando este conjunto de bateria, cargador e inversor a equipos provistos por la marca Victron Energy [2].

Esta tendencia fue la principal impulsora en el desarrollo de este trabajo, ya que fue necesario diseñar un dispositivo que permita acoplar los equipos provistos por la marca Victron a la red de domótica implementada por Enertik.

A su vez, se buscó desarrollar una aplicación que permita darle al usuario una opción adicional (ademas de la pantalla principal) que permita controlar todos los equipos dentro del vehículo, tanto los equipos de la red de Enertik como así también a los equipos de la red Victron.

De esta manera se buscó solucionar dos problemáticas presentes: por un lado, la migración hacia nuevas tecnologías energéticas por parte de los fabricantes de motorhomes y, por otro, la necesidad de brindar a los usuarios nuevas herramientas para la supervisión y el control del vehículo.

1.3. Estado del arte

En la actualidad, Enertik dispone de una estructura bastante bien definida a la hora de domotizar un motorhome. Entre los distintos dispositivos que componen esta red, se destaca uno en particular denominado PRC10 [3].

La figura 1.1 muestra un esquema básico de cómo se compone la red de domótica en la actualidad.

Como se puede apreciar, la arquitectura se basa en un esquema centralizado, donde el módulo PRC10 actúa como unidad de control principal del sistema. Los diferentes dispositivos presentes en el vehículo se comunican con este módulo, que se encarga de procesar la información recibida y ejecutar las acciones correspondientes sobre los distintos subsistemas.

También se puede observar la presencia de una pantalla táctil que actúa como interfaz de usuario entre el sistema de domótica y los ocupantes del vehículo. A través de esta interfaz es posible visualizar diferentes parámetros obtenidos a partir de los sensores distribuidos en la unidad, tales como valores asociados al sistema energético, estados de funcionamiento de los dispositivos conectados y otras variables de interés para el usuario.

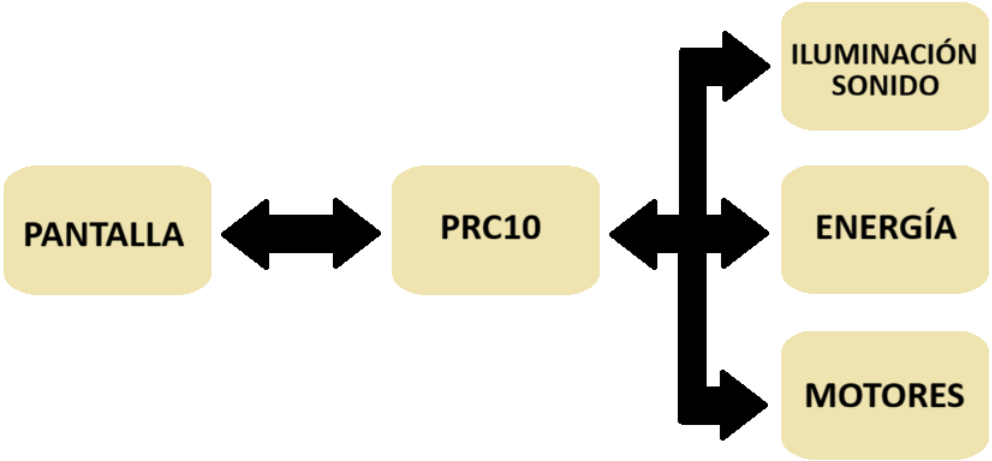

FIGURA 1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.

Además de permitir la supervisión del estado general del sistema, la pantalla brinda la posibilidad de interactuar con los distintos periféricos presentes en el motorhome. De esta manera, el usuario puede realizar acciones de control sobre diversos dispositivos, como la iluminación, los mecanismos motorizados y otros subsistemas integrados dentro de la red de domótica.

Entre los distintos dispositivos que componen la red de domótica de Enertik, se destacan los siguientes:

- PRC10: módulo central de procesamiento, responsable de la recepción de información proveniente de los demás dispositivos y de la toma de decisiones sobre el funcionamiento del sistema. Además, permite el comando de distintos periféricos, tales como la iluminación interna del vehículo, la apertura y cierre de esclusas de agua, la activación de la bomba de agua y el accionamiento de los motores asociados a las partes móviles del vehículo, como la mesa, la cama levadiza y el toldo.
- Pantalla HMI (*Human Machine Interface*): interfaz de usuario de tipo táctil que ejecuta el software principal del sistema. Permite la visualización del estado de las variables y la operación de los distintos dispositivos conectados a la red.
- Módulo transceiver: dispositivo que permite la adaptación de la pantalla HMI a la red de comunicación del motorhome.
- Módulo sensor de gases: dispositivo encargado de la medición de la composición de gases en el ambiente. Genera alertas ante la detección de condiciones perjudiciales para la salud.
- Módulo RGB: módulo de potencia destinado al control de la iluminación RGB distribuida en el vehículo.
- Módulo de control de patas: dispositivo responsable del accionamiento del sistema de estabilización del vehículo mediante el control de las patas de apoyo.
- Módulo de control de escalón: dispositivo responsable de la operación del mecanismo del escalón de acceso al vehículo.

- Módulo de relés para luces vehiculares: dispositivo encargado de la adaptación de niveles de tensión para el control de las luces del vehículo, tales como luces delanteras, traseras y señalización asociada al freno de mano.
- Módulo medidor de tanques de agua: módulo encargado de medir y reportar el nivel de llenado de los tanques de agua limpia, gris y negra.

1.4. Objetivos y alcances

El objetivo del presente trabajo es el desarrollo de un dispositivo que permita satisfacer dos necesidades principales: por un lado, la incorporación de nuevas tecnologías a la red de domótica desarrollada por Enertik y, por otro, brindar a los usuarios una opción adicional para controlar dicha red mediante una aplicación ejecutable en dispositivos móviles, tales como teléfonos celulares y tablets.

El microcontrolador elegido para el desarrollo del dispositivo fue el ESP32 [4]. Cabe destacar que este equipo no existía al momento de la planificación inicial del trabajo, por lo que su diseño fue realizado con la intención de que pueda convertirse posteriormente en un producto comercial para la empresa.

La figura 1.2 muestra el esquema de la arquitectura de comunicación implementada, en la que se destaca la integración del dispositivo basado en ESP32 como intermediario entre el módulo PRC10, el Cerbo GX y la aplicación móvil.

Dentro del ecosistema Victron Energy existe un dispositivo denominado Cerbo GX [5]. Este dispositivo actúa como módulo de control principal de la red de equipos Victron. En este contexto, el dispositivo desarrollado en este trabajo cumple la función de intermediario entre el módulo Cerbo GX y el módulo PRC10 perteneciente a la red de domótica de Enertik.

De esta manera, el módulo PRC10 puede acceder a la información generada por los equipos Victron y, a su vez, controlar ciertas variables que el sistema pone a disposición a través del módulo Cerbo GX.

Para lograr esta integración fue necesario que el dispositivo de comunicación sea capaz de manejar diferentes protocolos de comunicación.

En primer lugar, se incorporó el protocolo CAN (*Controller Area Network*) [6], ya que este es el protocolo utilizado en la red de domótica de Enertik. Por otra parte, Victron pone a disposición documentación técnica en su sitio web [7], donde se describe la forma de controlar el módulo Cerbo GX mediante el protocolo Modbus TCP [8].

Adicionalmente, el dispositivo debía contar con un terminal de cuatro hilos (+12 V, GND, CAN H y CAN L) con el objetivo de permitir su conexión directa al mismo cableado utilizado por la red de domótica. De esta manera, se garantiza tanto su alimentación como el intercambio de datos.

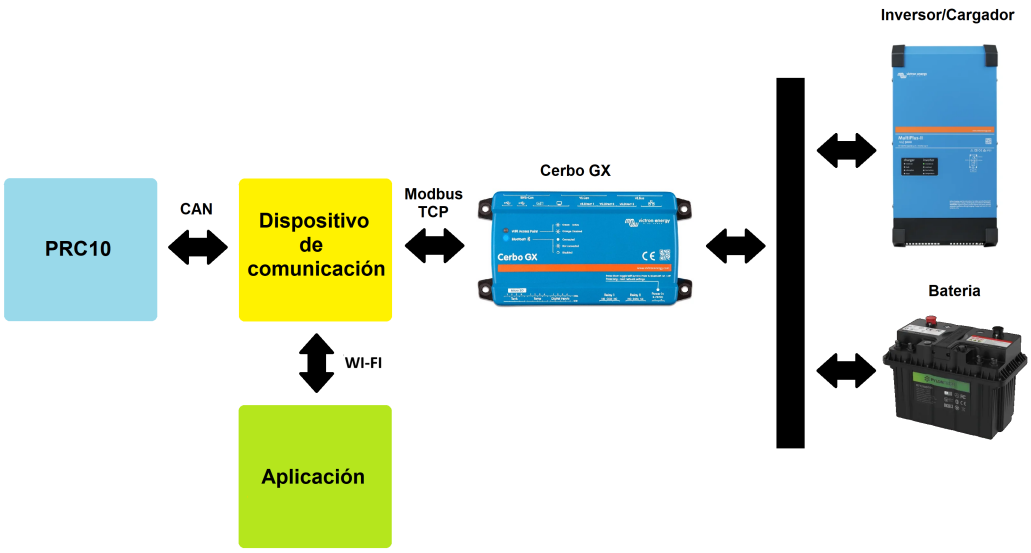

FIGURA 1.2. Arquitectura de comunicación del sistema con el ESP32 como intermediario entre PRC10, Cerbo GX y la aplicación móvil.

En cuanto a la aplicación móvil, se buscó que pueda ejecutarse en dispositivos portátiles y que sea capaz de comunicarse con el módulo PRC10 para obtener y mostrar en pantalla las lecturas de las distintas variables del vehículo, así como también enviar comandos de control hacia los dispositivos conectados a la red.

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación empleados en el desarrollo del trabajo, los componentes de hardware implementados para cumplir con los objetivos planteados y las herramientas utilizadas para el desarrollo del software.

2.1. Protocolos de comunicación

Como se mencionó en la sección 1.4, el dispositivo debía cumplir con la funcionalidad de actuar como intermediario entre las redes establecidas por Enertik y Victron, así como también constituir un punto de acceso para que la aplicación móvil pueda operar sobre el sistema.

Para lograr este objetivo, fue necesario implementar distintos protocolos de comunicación que permitan el intercambio de información entre los subsistemas involucrados, incluyendo CAN, Modbus TCP y una interfaz basada en HTTP.

2.1.1. Protocolo CAN

El protocolo CAN es un protocolo serie ampliamente utilizado en la industria automotriz. Permite la comunicación entre múltiples dispositivos sin la necesidad de un controlador central, ofreciendo un intercambio de datos rápido y confiable.

En este protocolo, la transmisión de datos se realiza a través de un par de conductores denominado CAN H y CAN L, utilizando señalización diferencial. Esta técnica consiste en transmitir la información como la diferencia de tensión entre ambos conductores, lo que proporciona una alta inmunidad a interferencias electromagnéticas, ya que el ruido externo tiende a afectar a ambos cables de manera similar y, por lo tanto, es cancelado al evaluar la diferencia entre ellos.

En cuanto a la representación lógica, un bit dominante (0 lógico) se caracteriza por una diferencia de tensión significativa entre CAN H y CAN L (normalmente un conductor adquiere un valor de tensión igual a la fuente y el otro a GND), mientras que un bit recesivo (1 lógico) se corresponde con un estado en el que ambos conductores presentan niveles de tensión similares.

Por otra parte, la comunicación en una red CAN se basa en un esquema de difusión, donde todos los nodos reciben los mensajes transmitidos. Cada mensaje incluye un identificador que permite a los dispositivos determinar si la información es relevante para ellos, ignorando aquellos mensajes que no corresponden a su funcionamiento.

2.1.2. Protocolo Modbus TCP

El protocolo Modbus es uno de los protocolos mas utilizados en la industria de la automatización. Su función principal es permitir que distintos dispositivos electrónicos se comuniquen entre sí de una forma sencilla y entandarizada.

En una red Modbus, exite una relación maestro-esclavo entre los dispositivos. El dispositivo maestro es el encargado de iniciar una comunicación y solicitar información. El esclavo se encarga de atender estas solicitudes y entregar los datos que el maestro requiera.

En el caso de Modbus TCP, este protocolo se implementa sobre redes Ethernet, utilizando el protocolo TCP/IP como medio de transporte. Esto permite que los dispositivos puedan comunicarse a través de redes locales, identificándose mediante direcciones IP.

La comunicación se basa en un esquema de tipo *request-response*, donde el cliente (maestro) envía una solicitud a un servidor (esclavo), y este último procesa la petición y devuelve una respuesta. Dentro de estas solicitudes se incluyen funciones específicas que permiten leer o escribir distintos tipos de registros, tales como *holding registers* o *input registers*.

En el contexto de este trabajo, el protocolo Modbus TCP es utilizado para establecer la comunicación con el módulo Cerbo GX, que expone diferentes registros que pueden ser consultados y controlados de forma remota.

2.1.3. Solicitudes HTTP

El protocolo HTTP (*HyperText Transfer Protocol*) [9] es un protocolo de aplicación ampliamente utilizado para la comunicación entre clientes y servidores. Su funcionamiento se basa en un modelo de tipo *request-response*, donde un cliente envía una solicitud (request) a un servidor, este procesa la petición y devuelve una respuesta (response) con la información requerida.

Las solicitudes HTTP permiten a los clientes acceder a recursos, enviar información y ejecutar acciones sobre un servidor. Estas peticiones están compuestas por distintos elementos, tales como el método de la solicitud, la dirección del recurso solicitado y, en algunos casos, un cuerpo de datos que contiene la información a enviar.

Existen diferentes tipos de métodos HTTP, cada uno con una función específica. Entre los más utilizados se encuentran el método GET, empleado para solicitar información al servidor, y el método POST, utilizado para enviar datos y generar cambios en el estado del sistema. Otros métodos relevantes incluyen PUT, orientado a la actualización de recursos, y DELETE, destinado a su eliminación.

En el contexto de este trabajo, las solicitudes HTTP son utilizadas como mecanismo de comunicación entre la aplicación móvil y el dispositivo basado en ESP32, lo que permite el envío de comandos de control y la obtención de información del sistema.

2.2. Componentes de hardware

Para el desarrollo de hardware fue necesaria la utilización de ciertos componentes. El principal de ellos fue el ESP32 como microcontrolador del equipo. A su vez, para poder adaptarlo a los distintos protocolos de comunicación, fue necesario el agregado de dos periféricos adicionales: el módulo W5110 y el MCP2551.

2.2.1. Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por la empresa Espressif Systems. Se trata de un dispositivo SoC (*System on Chip*) que integra múltiples periféricos y capacidades de procesamiento en un único dispositivo.

Entre sus principales características, el ESP32 dispone de puertos de propósito general (GPIO), conversores analógico-digitales (ADC), módulos de comunicación serie como UART, SPI e I^2C, temporizadores y periféricos para generación de señales PWM.

Además, el dispositivo incorpora conectividad inalámbrica, incluyendo Wi-Fi y Bluetooth, lo que lo convierte en una plataforma adecuada para el desarrollo de aplicaciones orientadas al campo IoT (*Internet of Things*). Todas las características mencionadas anteriormente vuelven al ESP32 en una opción mas que adecuada para el desarrolo del trabajo

FIGURA 2.1. Microcontrolador ESP32.

2.2.2. Módulo transceiver

Todos los dispositivos que pertenecen a la red de domótica de Enertik se conectan a través de un cableado de cuatro conductores (+12 V, GND, CAN H y CAN L). Este esquema permite tanto la alimentación como la comunicación de los equipos conectados a la red de manera sencilla.

Por otra parte, el protocolo de comunicación empleado corresponde a una biblioteca propia desarrollada por Enertik, basada en el estándar CAN. Para adaptar el ESP32 a esta red y permitir su correcta integración, fue necesaria la implementación de un módulo transceiver.

Este módulo actúa como interfaz entre el ESP32 y la red de domótica, y está compuesto por un microcontrolador PIC [10], en el que se implementa la biblioteca de comunicación, y un transceptor CAN MCP2551 [11], encargado de la adaptación de niveles eléctricos para la transmisión diferencial sobre el bus.

Como se puede apreciar en la figura 2.2, este módulo realiza la conversión de la red CAN, proveniente del sistema de domótica, a una interfaz de comunicación serial UART, compatible con el ESP32.

Esta solución permite desacoplar la gestión del protocolo de comunicación del procesamiento principal del sistema, lo que simplifica el desarrollo del firmware y facilita la integración del dispositivo en la red de domótica existente.

FIGURA 2.2. Funcionamiento del módulo transceiver.

2.2.3. Módulo Ethernet W5100

Para lograr la comunicación mediante el protocolo Modbus TCP, fue necesaria la utilización de un módulo Ethernet basado en el circuito W5100 [12].

El W5100 es un controlador Ethernet que permite dotar de conectividad de red a sistemas embebidos, gestionando internamente la pila de protocolos TCP/IP. La comunicación con el microcontrolador se realiza a través de una interfaz SPI, lo que facilita su integración en plataformas como el ESP32.

Mediante el uso de este módulo, el dispositivo desarrollado puede establecer conexiones sobre redes Ethernet, lo que permite la implementación del protocolo Modbus TCP para la comunicación con equipos del ecosistema Victron.

FIGURA 2.3. Módulo W5100.

2.3. Herramientas de software

Para el desarrollo del software fue necesario definir tanto el entorno de desarrollo integrado (IDE) como el lenguaje de programación a emplear. Asimismo, se evaluaron las herramientas necesarias para el desarrollo de la aplicación móvil.

2.3.1. Entorno de desarrollo Arduino

El microcontrolador ESP32 es compatible con el entorno de desarrollo Arduino [13], lo que permitió utilizar este IDE (*Integrated Development Environment*) como plataforma principal para el desarrollo del firmware. La elección de este entorno se basó en su facilidad de uso, amplia documentación y la disponibilidad de una gran cantidad de bibliotecas previamente desarrolladas.

En particular, el uso del IDE Arduino facilitó la implementación de los distintos protocolos de comunicación requeridos en el proyecto, tales como HTTP y Modbus TCP. Para este último, se emplearon bibliotecas específicas que fueron adaptadas para su funcionamiento sobre el módulo Ethernet W5100.

Adicionalmente, este entorno permite una rápida compilación y carga de programas sobre el dispositivo, así como el uso de herramientas de depuración básicas, lo que lo convierte en una opción adecuada para el desarrollo y prueba del sistema.

Para la implementación de la comunicación Modbus TCP, se utilizó una biblioteca desarrollada por Andre Sarmento Barbosa y Alexander Emelianov [14]. En conjunto con la biblioteca Ethernet provista por el entorno Arduino, esta herramienta permite la implementación del protocolo Modbus en distintas variantes, entre ellas Modbus TCP.

Para el desarrollo de la comunicación UART y del servidor interno en el ESP32, se emplearon las bibliotecas estándar provistas por defecto en el entorno de desarrollo.

2.3.2. Funcionamiento del ESP32

Para la programación del microcontrolador se optó por el lenguaje C++, debido a que es el lenguaje en el que se encuentran desarrolladas la mayoría de las bibliotecas disponibles en el entorno Arduino, lo que facilitó la integración de distintas funcionalidades requeridas en el proyecto.

Como metodología de desarrollo, se utilizó FreeRTOS [15], que se encuentra integrado en el ESP32. Este sistema operativo en tiempo real permite organizar el firmware mediante la ejecución de múltiples tareas, cuya planificación da lugar a un comportamiento concurrente a nivel lógico. Esto mejora la estructura del programa y facilita la gestión de distintas funcionalidades en paralelo.

Adicionalmente, FreeRTOS dispone de un planificador (*scheduler*) encargado de coordinar la ejecución de cada tarea. Este asigna el tiempo de procesamiento en función de su prioridad. Asimismo, proporciona mecanismos que permiten la comunicación y sincronización entre tareas, lo que facilita la correcta coordinación de las distintas funcionalidades implementadas en el sistema.

En este contexto, el firmware fue dividido en distintas tareas principales. En primer lugar, se implementó una tarea encargada de la gestión de la comunicación Wi-Fi, donde el ESP32 opera en modo *Access Point*, permitiendo la conexión de la aplicación móvil y actuando como servidor HTTP para la recepción de solicitudes.

Por otra parte, se desarrolló una tarea dedicada a la recepción de mensajes provenientes de la red CAN, que se encarga de procesar la información recibida desde los distintos dispositivos conectados a la red de domótica.

Finalmente, se implementó una tarea central de procesamiento, cuya función es interpretar la información recibida tanto desde la red CAN como desde la interfaz HTTP, y tomar las decisiones correspondientes para el control del sistema.

2.3.3. Desarrollo de la aplicación móvil

Para el desarrollo de la aplicación móvil se optó por utilizar un conjunto de herramientas que trabajan de forma integrada. La figura 2.4 presenta un esquema de la relación entre las distintas tecnologías empleadas.

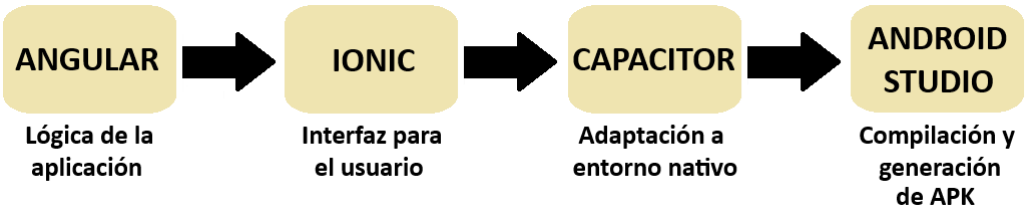

FIGURA 2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.

Se utilizó el framework Ionic [16] como base para el desarrollo de la aplicación, debido a que permite la creación de interfaces móviles multiplataforma a partir de tecnologías web.

Ionic se apoya en el framework Angular [17], que proporciona una estructura robusta para el desarrollo de interfaces de usuario dinámicas y organizadas en componentes. Angular utiliza lenguajes como TypeScript, HTML y CSS, lo que facilita la construcción de aplicaciones modulares y escalables.

Para la adaptación de la aplicación a dispositivos móviles se utilizó Capacitor [18], herramienta que permite empaquetar la aplicación web como una aplicación nativa y facilita el acceso a funcionalidades propias del dispositivo.

Finalmente, se empleó Android Studio [19] para la compilación y generación del archivo APK (*Android Package Kit*), que constituye el paquete instalable de la aplicación en sistemas Android. Este archivo contiene todos los recursos necesarios para su ejecución, tales como el código compilado, las bibliotecas utilizadas y los archivos de configuración.

Ionic dispone de un amplio conjunto de bibliotecas que pueden ser utilizadas en el desarrollo de aplicaciones. Entre ellas, se encuentra una biblioteca destinada a la gestión de solicitudes HTTP. En este contexto, se optó por utilizar dicha biblioteca como base para el desarrollo de funciones propias orientadas a la comunicación entre la aplicación y el ESP32.

Como se mencionó en la sección 2.1.3, la solicitud GET es utilizada para la realización de peticiones destinadas a la obtención de datos desde el servidor. A través de este mecanismo es posible consultar el estado de las distintas variables del sistema en el motorhome.

Por otra parte, mediante solicitudes POST es posible enviar datos desde la aplicación hacia el ESP32. Esto permite el control de las distintas variables del sistema y la ejecución de comandos sobre los dispositivos.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno lstlisting con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
 las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código 3.1:

```
#define MAX_SENSOR_NUMBER 3
#define MAX_ALARM_NUMBER  6
#define MAX_ACTUATOR_NUMBER 6

uint32_t sensorValue[MAX_SENSOR_NUMBER];
FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
state_t alarmState[MAX_ALARM_NUMBER];           //ON or OFF
state_t actuatorState[MAX_ACTUATOR_NUMBER];     //ON or OFF

void vControl() {

  initGlobalVariables();

  period = 500 ms;

  while(1) {

    ticks = xTaskGetTickCount();

    updateSensors();

    updateAlarms();

    controlActuators();

    vTaskDelayUntil(&ticks, period);
  }
```

```
28 }
```

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se puedo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

[1] Enertik Argentina. Última consulta: 2025-03-14. URL: https://enertik.com/ar/.

[2] Victron Energy. Última consulta: 2025-03-14. URL: https://www.victronenergy.com.es/.

[3] Enertik Argentina. *PRC10: manual para el usuario.*

[4] Espressif. Última consulta: 2025-03-21. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html.

[5] *Cerbo GX*. Última consulta: 2025-03-16. URL: https://www.victronenergy.com.es/communication-centres/cerbo-gx.

[6] *CAN Bus Explained - A Simple Intro [2025]*. Última consulta: 2025-03-16. URL: https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial.

[7] Manual de GX - Modbus TCP. Última consulta: 2025-03-15. URL: https://www.victronenergy.com/live/ccgx:modbustcp_faq.

[8] *Modbus: Qué es y cómo funciona*. Última consulta: 2025-03-19. URL: https://www.cursosaula21.com/modbus-que-es-y-como-funciona/.

[9] *Overview of HTTP*. Última consulta: 2025-03-19. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview.

[10] *PIC18F66K80 FAMILY*. Última consulta: 2025-03-19. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/PIC18F66K80%20FAMILY%20Enhanced%20Flash%20MCU%20with%20ECAN%20XLP%20Technology%2030009977G.pdf.

[11] *MCP2551*. Última consulta: 2025-03-19. URL: https://ww1.microchip.com/downloads/en/devicedoc/20001667g.pdf.

[12] *W5100*. Última consulta: 2025-03-19. URL: https://cdn.sparkfun.com/assets/e/2/f/c/2/W5100_Datasheet_v1_1_6.pdf.

[13] *Arduino*. Última consulta: 2025-03-25. URL: https://docs.arduino.cc/software/ide-v1/tutorials/PortableIDE/.

[14] *Github: modbus-esp8266*. Última consulta: 2025-03-25. URL: https://github.com/emelianov/modbus-esp8266.

[15] *FreeRTOS*. Última consulta: 2025-03-19. URL: https://www.freertos.org/.

[16] *Introduction to Ionic*. Última consulta: 2025-03-19. URL: https://ionicframework.com/docs.

[17] *Arduino*. Última consulta: 2025-03-25. URL: https://angular.dev/.

[18] *Capacitor web*. Última consulta: 2025-03-25. URL: https://capacitorjs.com/.

[19] *Android Studio*. Última consulta: 2025-03-25. URL: https://developer.android.com/studio?hl=es-419.